

Affora

A Design System for Agent-Friendly Interfaces

JIN GAO, Independent Researcher, USA

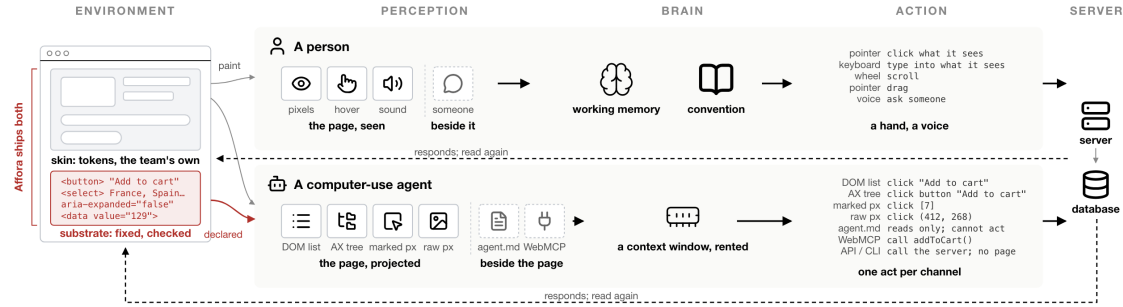


Fig. 1. **One interface, two kinds of reader.** People and agents may act on the same application while receiving different representations of its controls, state, and possible actions.

Computer-use agents increasingly operate software designed for people, but interfaces often leave actions or task state unclear to machine readers. We present Affora, a design system that supports both readers while preserving visual freedom and familiar human workflows. Three controlled studies examine component implementations, visual variation, and interaction-design principles. Their findings inform guidance from individual components to complete sites, supported by reusable implementations and executable checks. Agent performance depends on the interaction meaning available through its interface representation; substantial visual variation remains possible when that meaning is preserved. Evaluation on independently authored interfaces shows gains where Affora addresses existing deficits, but limited effects where those deficits are absent or outside its coverage. A workflow case provides preliminary evidence of reduced interaction cost. Affora connects user experience and agent experience through a shared interface rather than a separate agent-only surface.

CCS Concepts: • **Human-centered computing** → **HCI design and evaluation methods**; *Interaction paradigms*; *Empirical studies in HCI*.

Additional Key Words and Phrases: design systems, agent experience design (AX), agent-computer interaction, computer-use agents, affordances, shared human-agent interfaces, user interface design

1. Introduction

Computer-use agents increasingly operate software interfaces used for everyday tasks. Their performance depends not only on model capability but also on how an interface represents its available actions and current state. A control may appear obvious to a person yet be absent or ambiguous in the representation

an agent receives. Improving agent reliability therefore raises a design question: how should the interface itself support an agent reader while remaining familiar to people?

We present **Affora**, a design system for a shared human-agent interface. Its name draws on J. J. Gibson’s concept of *affordance*, understood as possibilities for action in relation to an actor’s capabilities (Gibson 1977). Affora specifies how interaction meaning should remain recoverable from the interface while allowing visual expression to vary. In the web implementation studied here, this structure is carried by the rendered DOM and exposed through the agent’s observation channel. The system is intended to fit existing design and development workflows while retaining familiar interfaces for human users.

We investigate this approach through three studies of component implementations, visual variation, and established interaction-design principles. Their findings inform design guidance across component, layout, flow, and site scales, implemented through reusable components and executable checks. We then evaluate transfer to independently authored interfaces and examine interaction cost in a complete workflow. The contribution is a design system together with evidence about where its rules help, where they have little effect, and where their coverage remains incomplete. Affora connects user experience (UX) and agent experience (AX) within one interface; its benefits for human users remain a subject for future evaluation.

2. Related work

Design systems, accessibility, and semantic interfaces. Design systems distribute tokens, components, and interaction patterns to preserve consistency and accessibility across products (Putnam, Rose, and MacDonald 2023; Lamine and Cheng 2022). Accessibility standards such as WCAG and ARIA likewise specify how controls, names, relationships, and states should be exposed beyond their visual presentation. These traditions establish that an interface has structure beneath its styling, but they optimize primarily for human use, including use through assistive technologies. Affora builds on that semantic foundation while asking a different question: whether the rendered interface exposes enough persistent, enumerable, and operable structure for an autonomous reader to complete a task. Agents and disabled users are not equated; rather, both make the consequences of an underspecified semantic layer visible (Reid 2024).

Computer-use agents and interface grounding. Research on computer-use agents has largely asked how an agent can understand and act on interfaces as they are given. Mind2Web, WebArena, VisualWebArena, SeeAct, WebVoyager, BrowserGym, and related systems study grounding through DOM representations, accessibility trees, screenshots, or combinations of these representations (Deng et al. 2023; Zhou et al. 2024; Koh et al. 2024; Zheng et al. 2024; He et al. 2024; Le Sellier De Chezelles et al. 2024). Set-of-Mark prompting similarly alters the agent’s visual observation to make targets easier to reference (Yang et al. 2023). Affora reverses the direction of intervention: rather than adapting the reader to an existing interface, it asks which properties of the interface itself should be invariant so that multiple kinds of reader can operate it reliably.

Agent-facing representations and action layers. Agent-facing documentation and structured tools provide additional ways for agents to understand or operate software. Files such as `llms.txt` and project instruction files provide information outside the rendered interface (Howard 2024), while WebMCP exposes callable operations from web applications (Walderman et al. 2026). These mechanisms differ in what they make available to the agent and need not replace the graphical interface. Affora focuses on the structure exposed

by the interface itself. Section 6.2 compares concrete implementations on targeted interface failures; Table 1 describes those interventions rather than guarantees of the underlying standards.

Table 1. **Representations and operations exposed by the evaluated interventions.** Entries describe the implementations used in our comparison, not universal properties of accessibility standards, instruction files, or tool protocols.

Intervention	What the agent receives	Where actions execute
ARIA / structured data	Additional interface metadata	Existing interface
Instruction file	Separate textual guidance	Existing interface
WebMCP-style tools	Tool descriptions and callable operations	Tool interface
Affora	Revised shared-interface structure	Revised interface

Agent experience and agent-compatible interface design. Recent work has begun to treat computer-use agents as a distinct class of interface reader. Goldenberg and Goldenberg discuss *agent experience* by reconsidering Nielsen’s usability heuristics for GenAI agents (Goldenberg and Goldenberg 2025). Rongon et al. propose a framework for agent-readable interfaces that emphasizes observable state and explicit interaction meaning (Rongon, Hasan, and Prangon 2026). Liu et al. experimentally evaluate augmentations of Nielsen’s heuristics, finding improved agent task completion and no observable usability regressions in their human evaluation (Liu et al. 2026). Together, these works establish interface design itself as a variable in reliable computer use.

Affora extends this direction by expressing agent compatibility through reusable implementations and checks within a design system. It tests principles drawn from multiple interaction-design traditions under readers with different perceptual, memory, and action costs, and applies the resulting guidance across component, layout, flow, and site scales. Section 4.3 examines these differences experimentally.

3. Research Questions and Interface Model

Affora is motivated by a simple question: *what properties must an interface expose for a computer-use agent to operate it reliably, without constraining how it looks to people?* We address this through three research questions:

RQ1: Where does agent-facing interface failure arise? When computer-use agents fail on ordinary interface components, to what extent is the failure explained by the interface’s semantic substrate rather than by model capability or visual presentation?

RQ2: Which visual design decisions can vary without reducing agent performance? If the semantic substrate is held fixed, how freely can designers vary styling and layout while preserving reliable agent interaction?

RQ3: How do established interaction-design principles change when the reader is an agent? Which principles developed for human users still hold, which become more important, and which weaken or reverse when the reader has different perceptual, memory, and action costs?

The three studies in Section 4 address these questions in order. Study 1 compares component implementations to localize the deficit; Study 2 holds the substrate fixed while varying visual design; and Study 3 experimentally re-examines established interaction-design principles for computer-use agents.

A shared interface, two readers. A software interface presents more than what it visually renders. We distinguish between what an interface **paints** and what it **declares**. The painted layer contains its visual expression—colour, typography, shape, spacing, composition, motion, and other styling decisions. The declared layer contains the controls, names, relationships, state, choices, and task-relevant information represented in a form that can be recovered from the interface itself. We call this declared layer the **semantic substrate**, or simply the *substrate* below.

People and computer-use agents can operate the same interface while reading these layers differently. A person primarily perceives the rendered interface and interprets its text, spatial organization, visual hierarchy, and learned conventions. An agent instead acts through a projection of the interface, such as a serialised list of interactive elements with roles and names (Zheng et al. 2024; Lù, Kasner, and Reddy 2024), an accessibility tree (Zhou et al. 2024; Le Sellier De Chezelles et al. 2024), a marked screenshot, or raw pixels. These projections expose different portions of the interface and different action spaces. DOM, accessibility-tree, and marked-screen representations explicitly enumerate candidate actions, whereas raw-pixel interaction requires the agent to infer actionable regions visually.

The substrate matters at multiple scales. At component scale, it determines whether an apparent control is represented as an operable control, whether it has a stable name, whether its current state can be recovered, and whether its available choices are exposed. At layout scale, it preserves recoverable relationships among components as their composition changes. At flow scale, it determines whether the interface carries the state and progress needed to continue a task. At site scale, it includes stable mappings between functions, names, and representations across pages. Our use of *substrate* is narrower than the term’s use in interaction theory for the computational medium in which information resides (Beaudouin-Lafon, Bødker, and Mackay 2021): here it denotes the machine-readable structure carried by the rendered interface itself.

Figure 1 makes this distinction concrete. The same rendered interface may provide rich visual cues to a person while exposing only part of its actionable structure to a machine reader. Conversely, an agent-facing channel such as `agent.md` or WebMCP may provide information or actions that do not belong to the shared rendered interface at all. Affora focuses on the shared surface: the structure that remains part of the interface both readers use.

Figure 2 illustrates three representative substrate changes: making an apparent control directly operable, keeping choices available to the machine-readable representation, and retaining an outcome as persistent state. These interventions can alter what an agent can perceive and act upon without requiring a corresponding change in the interface’s visual identity.

The substrate is therefore not an agent-only representation. It remains part of the same interface rendered for a person, which creates substantial overlap with accessibility practice: both depend on controls, names, relationships, and state being represented beyond visual styling. Affora builds on that foundation but focuses specifically on whether the shared interface exposes the information a computer-use agent needs to identify available actions, recover task state, and continue interaction through the representations it receives.

In Norman’s terms (Norman 2008, 2013), visual appearance can provide a strong signifier to a person while providing little or no corresponding signifier in the representation available to an agent. Affora therefore treats

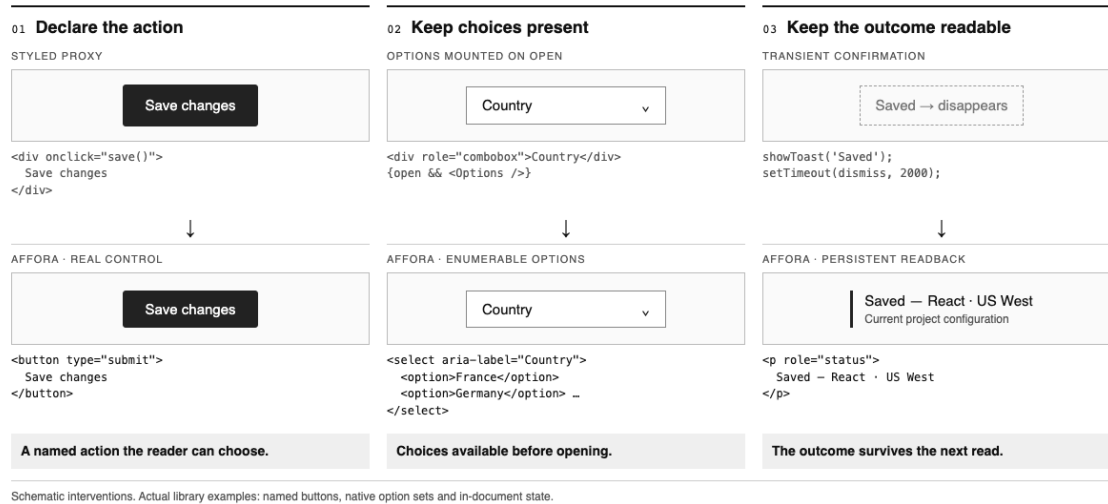


Fig. 2. **Examples of changes to the semantic substrate.** The visual interface can remain similar while its controls, choices, and state become more explicitly represented for machine readers.

agent-facing interface design as a problem of making action possibilities explicit across machine-readable representations while leaving the visual expression of the interface free to vary.

4. What Makes an Interface Agent-Friendly

We examine agent-friendliness as a property of the interface, rather than of agent capability alone. Three studies progressively isolate where it comes from, what can vary without disrupting it, and how it changes established interaction-design assumptions.

4.1 Study 1: Where interface failures arise. Design. We compare implementations of sixty interactive components using native HTML and eight commonly used component libraries, holding task content and goals constant. The libraries represent modern UI abstractions rather than systems designed for agents. We examine how each implementation exposes actionable structure after rendering; Figure 3 shows nine representative component kinds.

Main result. Agent performance varies systematically across implementations. Plain semantic HTML reaches 91%, while the eight component libraries range from 86.7% to 68.3% on the mid-tier model (Fig. 3). Failures cluster around recurring representational differences: choices that appear only after interaction, controls whose role or state is difficult to recover, and interaction outcomes that are weakly represented in the rendered structure. In a controlled repair sequence, correcting semantics raises success from 43% to 67%, and additionally exposing agent-relevant choices and state raises it to 90%.

Meaning. The result is not that modern component libraries are poorly designed, but that abstractions optimized for human-facing interfaces do not necessarily preserve the same information for a machine reader after rendering. Two implementations can present equivalent tasks to a person while exposing different

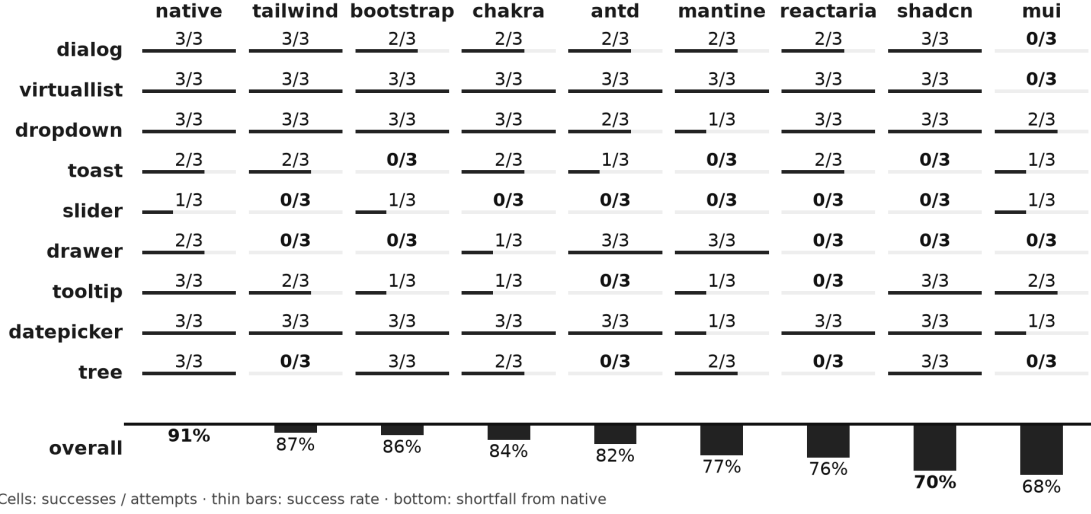


Fig. 3. **Agent success across component libraries and component kinds.** The same interaction tasks are implemented using native HTML and eight component libraries, revealing systematic differences in agent task completion despite equivalent task content. The matrix shows nine representative component kinds; aggregate results cover the full component set.

DOM structure, state, or action possibilities to an agent. Semantic correctness therefore matters, but is not sufficient: reliable agent interaction also depends on whether relevant controls, choices, and outcomes remain explicit and recoverable in the interface substrate.

4.2 Study 2: What visual design matters to agents. Design. We study visual design at two levels while holding task content and semantic substrate constant. First, five components are rendered across sixteen themes and six layout archetypes. Second, we vary individual visual properties on a signed scale from treatments that contradict familiar conventions, through neutral treatments, to treatments that reinforce them (Fig. 4). For this exploratory visual-cue analysis, we report complete five-condition blocks for the available Luna marked-pixel run; each block shares a component, property family, and repetition.

Main result. Across the tested themes, DOM-channel completion remains at 99.5%, matching the native presentation; pixel performance is 92%, compared with 94% for native controls. All episodes succeed in the tested extreme-theme and layout combinations in both channels. The exploratory visual-cue sweep is also close to ceiling: completion ranges from 98.7% to 99.8% across the five signed strengths, without a monotonic improvement as cues become more reinforcing. These results do not establish a general benefit from stronger visual cues.

Meaning. Within the tested components and tasks, substantial changes in visual identity preserve agent performance when the semantic substrate remains intact. The visual-cue sweep does not justify ranking aesthetics by agent-friendliness. Its marked-pixel reader also receives enumerated targets, so the result should not be generalized to agents that locate actions from raw pixels alone. Visual freedom is therefore supported within the tested observation models, rather than established for every kind of agent reader.

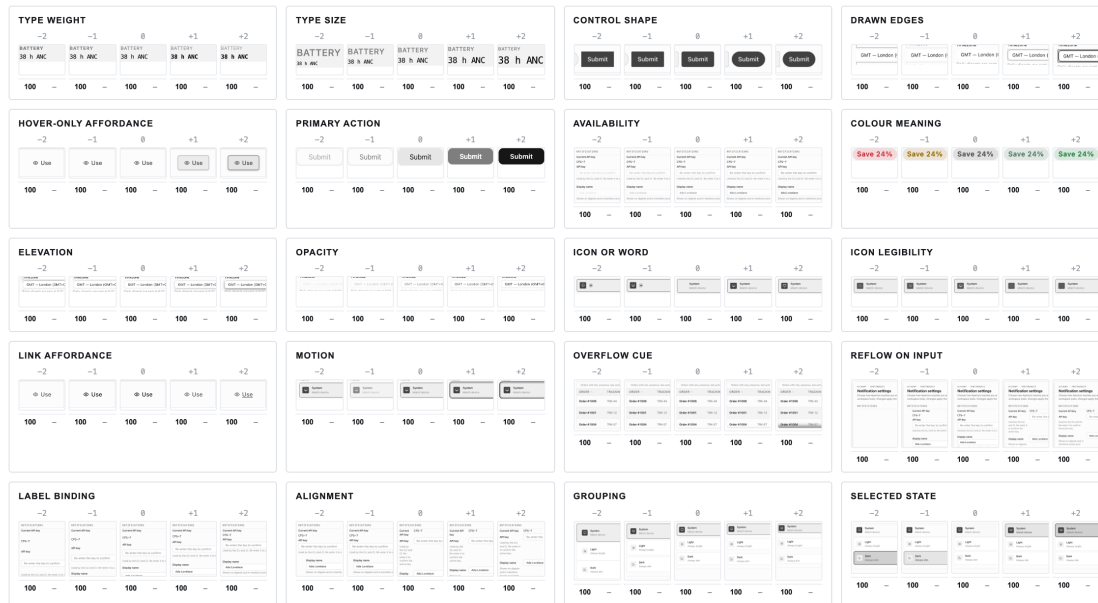


Fig. 4. **Visual properties from counter-signaling to reinforcing familiar affordances.** Each family varies one visual property while holding the semantic substrate and task content constant, providing exploratory comparisons of visual signals. Missing model results are not treated as failures.

4.3 Study 3: How canonical interaction-design principles change for computer-use agents. Design. We translate seven established interaction-design principles into controlled interface comparisons, drawing on Miller (1956), Nielsen (1994), Norman (2013), and Shneiderman et al. (2016), holding task information constant while varying the design decision implied by each principle. We measure task completion or interaction cost across three models and two observation channels, targeting ten repetitions per replication condition. Figure 5 illustrates conditions from the Luna runs, including the earlier vocabulary pilot. We interpret each principle in relation to the agent’s observation, memory, and action model.

Main result. The interaction-design canon does not transfer uniformly to computer-use agents. Explicit remedies improve recovery, while added confirmation increases interaction cost in the tested tasks. Recognition and visual signifiers depend on what the observation channel exposes. Splitting a form into steps adds interaction cost without a demonstrated completion benefit, and explaining disabled states shows no consistent gain across replications. These findings concern observed task behavior, not a direct measurement of cognitive load or the safety value of confirmation.

Meaning. The change of reader changes the mechanism behind familiar design advice. Human-facing principles often assume persistent memory, learned conventions, perceptual continuity, and meaningful motor cost. Computer-use agents may instead re-observe the interface at each step, lose task state across context boundaries, or receive candidate actions already enumerated by their observation channel. Consistency therefore matters when it preserves a mapping the agent must reuse, but less when every screen is interpreted

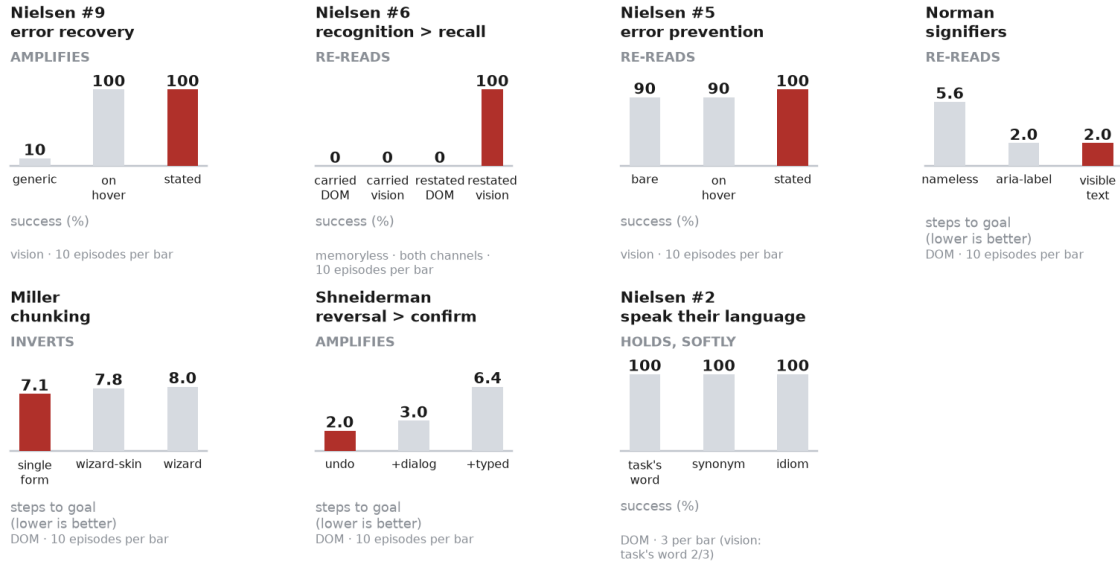


Fig. 5. Illustrative measurements for seven interaction-design principles. Panels show Luna conditions with the channel and sample size indicated, including an earlier vocabulary pilot; three-model replication results are not shown in this figure.

afresh. Recognition matters when the required information is present in the current observation, not merely because it appeared earlier. Confirmation can add unnecessary interaction cost, while explicit recovery becomes more valuable when the agent cannot infer a remedy from context. The implication is not to discard the existing canon, but to restate its principles in terms of the perceptual, memory, and action model of the reader.

5. An agent-friendly design system

From the studies, we derive a design methodology for agent-friendly interfaces: make action-relevant structure explicit and machine-readable, preserve visual expression as an independent design layer, and carry task state and interaction meaning beyond individual components. Affora operationalizes this methodology as a design system spanning component, layout, flow, and site scales. Across these scales, Affora follows a common set of design rules:

- **Make task-relevant structure explicit and persistent.** Controls, choices, state, and outcomes should remain available to the reader rather than existing only transiently or in interaction history.
- **Align machine-readable and visible structure.** What the interface exposes in the DOM or other machine-readable representations should correspond to what is visibly presented and operated.
- **Prefer presence over hidden disclosure.** Information needed for planning should remain discoverable without first requiring hover, expansion, animation, or other transient interaction.
- **Preserve semantic continuity.** Recurring functions, task state, and interaction meaning should remain identifiable as the reader moves across components, steps, and pages.

- **Keep visual expression flexible.** Colour, typography, shape, composition, and other stylistic decisions may vary as long as they do not remove or contradict the interaction meaning above.

Figure 6 summarizes how these rules are applied throughout the system.

5.1 Design scales. Component scale. Affora follows *substrate invariant, skin variable*: controls, names, state, choices, and outcomes remain explicit and operable, while colour, typography, shape, motion, and other stylistic properties may vary. Principles P1–P10 formalize these requirements.

Layout scale. Layout concerns how components are composed while preserving recoverable relationships and access to task-relevant content. It applies existing requirements, particularly P5 on semantic hierarchy and P9 on structural correspondence, across a page rather than introducing a separate family of principles. Layout transformations are assessed through page-level preservation checks and the theme–layout comparisons in Study 2. The evidence status of P5 and P9 remains observational; the layout experiment does not validate every possible composition.

Flow scale. The interface must carry the state a reader needs to continue a task, including progress, prior choices, requirements, recovery paths, and completion. Designers remain free to vary pacing, screen count, and presentation order. Principles F1–F7 and reusable flow patterns encode these requirements.

Site scale. Affora preserves mappings that a reader may need to reuse across pages, such as stable names and representations for recurring functions. It does not require visual uniformity across the product; the constraint is semantic continuity. Principles SC1–SC3 define this scope.

5.2 System artifacts. Affora is delivered as both a set of plug-and-play UI building blocks and a set of software design guidelines for agent-friendly interfaces. Teams can adopt the reference components directly or reproduce the same rules in their existing stack. The components separate semantic structure from visual expression, allowing Affora to adapt to different visual themes as long as the underlying design constraints are preserved.

Reusable UI components. Affora provides reference implementations for common interface elements, including forms, selection controls, dialogs, tables, and product-oriented components. Each embeds the required semantic substrate by default, so teams do not need to reconstruct agent-facing behavior for every interface.

Design guidance and theming. The library is paired with component-, layout-, flow-, and site-level design rules. Style and layout tokens allow colour, typography, shape, density, and composition to vary while keeping task-relevant controls, state, and relationships intact. Affora therefore defines constraints on interaction meaning rather than prescribing a single visual language.

AI-assisted implementation. Because the design rules are explicit and the reference components are distributed as source, they can be used directly in AI-assisted coding workflows. A coding agent can reuse established components and patterns rather than repeatedly inferring the intended semantics of custom interface code. By making controls, state, choices, and outcomes explicit by construction, Affora also removes a class of interface-induced ambiguities that would otherwise surface later as agent failures and debugging work.

Figure 6 relates the design scales to the system’s supporting mechanisms.

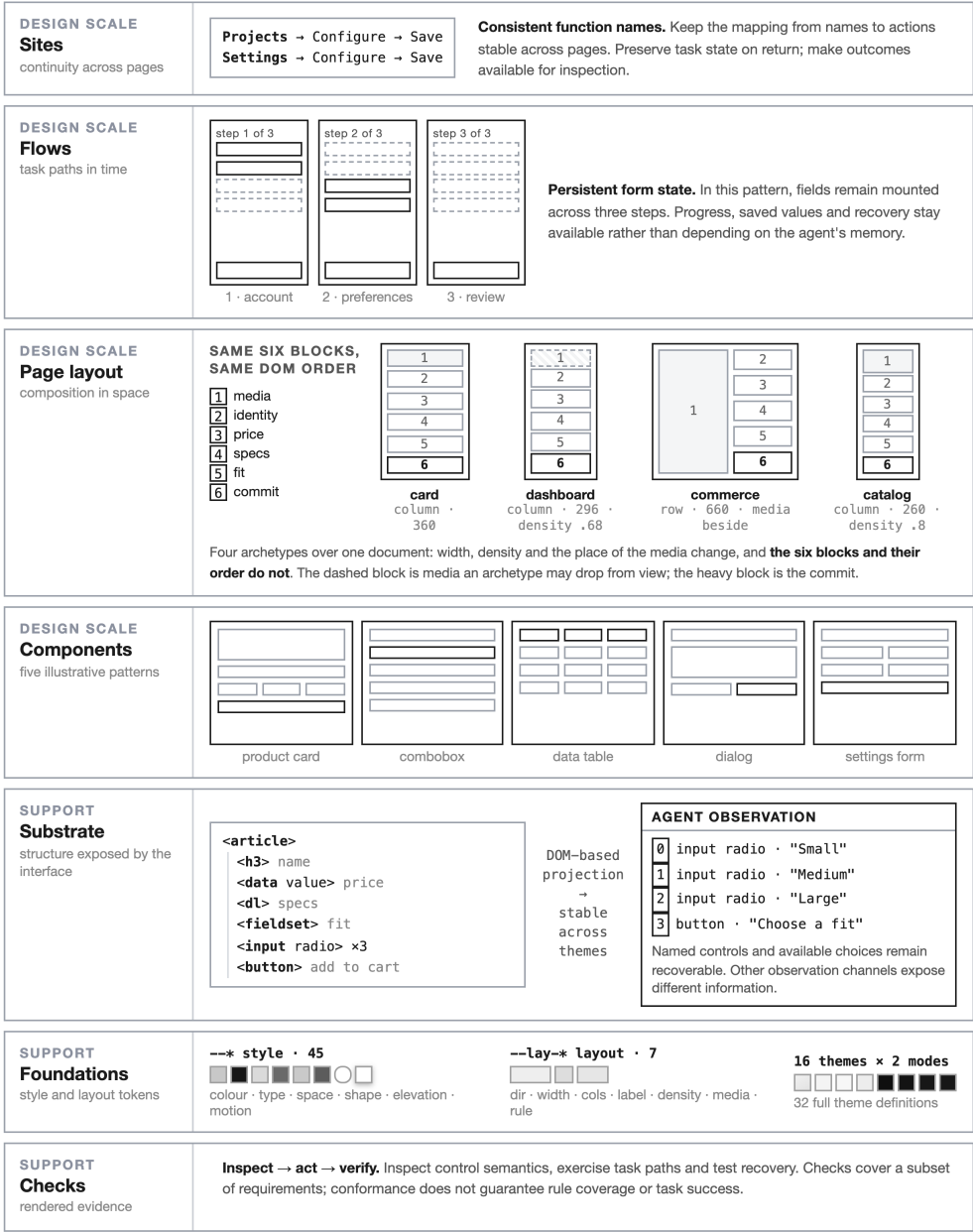
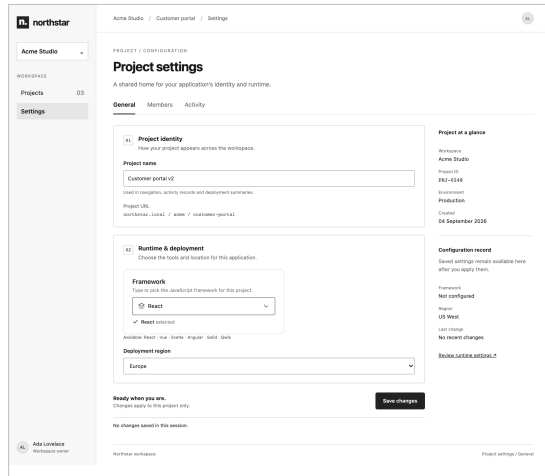


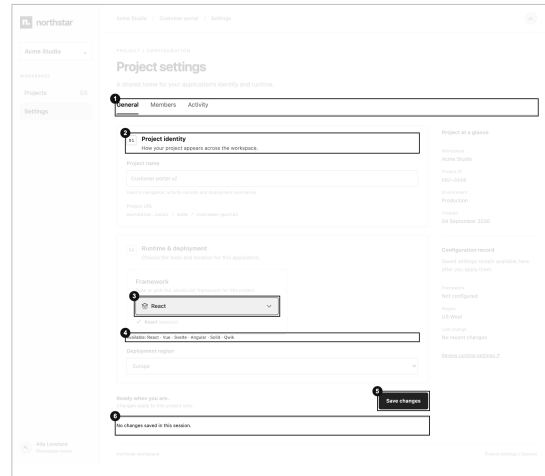
Fig. 6. **Affora across four design scales.** Component, layout, flow, and site guidance specifies how interaction meaning is preserved as interfaces are composed. The semantic substrate carries machine-readable structure, tokens support visual variation, and executable checks cover a subset of the design requirements.

Fig. 7 shows the system in a working application. Northstar is an author-built project-settings case using the Affora combobox and theme tokens. The same page appears with visual annotations and source excerpts; these are implementation examples, not additional performance measurements.

01 / Interface



02 / Design decisions



- 1 Current section stays visible.
- 2 Fields are grouped by task.
- 3 Input is bounded and labelled.
- 4 Choices are shown in text.
- 5 Commit action is prominent and named.
- 6 Feedback has a persistent location.

03 / Implementation excerpts

A / Named native input

```
<label htmlFor={inputId}>
  Framework
</label>
<input id={inputId}
  type="text"
  list={listId} ... />
```

A → labelled input (3)

B / Available choices

```
<datalist id={listId}>
  {FRAMEWORKS.map(f => (
    <option key={f.value}
      value={f.value}>
        {f.hint}
    </option>
  ))}
</datalist>
```

B → choices (4)

C / Persistent outcome

```
setSaved({name,
  framework, region});
setError('');

<div role="status" ...>
  Saved - {saved.name} ...
</div>
```

C → confirmation (6)

Fig. 7. Affora in a working application. The page, its visual affordances and its implementation.

5.3 From guidelines to executable constraints. Affora turns part of its design guidance into constraints that software can inspect directly. Component-level checks test whether controls are operable and named, state is recoverable, choices remain enumerable, and machine-readable targets align with the visible interface. Page-level checks additionally verify that transformations preserve information, interaction opportunities, and behavior.

These constraints can also be exposed in machine-readable form to AI-assisted development workflows. Coding agents can consume Affora’s rules alongside the component library—for example through project instructions or `skill.md`-style guidance—and apply them while generating, modifying, or debugging interfaces. This creates a path from post-hoc compliance checking toward enforcing agent-friendly structure during implementation itself.

Passing the checks is a conformance floor rather than a guarantee of task success: it establishes that required interface properties are present, while model capability, visual grounding, flow state, and application behavior may still affect performance. The full mapping from principles to checks and evidence status is given in the appendix, Principle index.

6. Evaluation

We evaluate whether shared-interface interventions address the same targeted failures as alternative agent-facing mechanisms, whether the rules transfer to independently authored interfaces, and how rule coverage differs from conformance. A separate workflow case examines interaction cost.

Execution harness and reproducibility. All reported episodes run through our authored agent-evaluation harness. It provides a common execution loop for computer use and structured tool use, ReAct-style observe–reason–act cycles, bounded retry after failed actions, and task-scoped memory across steps. We instantiate this harness with the OpenAI ChatGPT 5.6 Luna and ChatGPT 5.6 Terra variants, both at the low reasoning tier, referred to throughout as *Luna* and *Terra*. For every episode, the harness records the model and observation configuration, action and retry trace, terminal application state, and evaluator verdict. Experiments ran locally on macOS 26.2 on a MacBook Pro with an Apple M4 Pro and 48 GB unified memory, using Node.js v25.2.1, Python 3.14.7, and Docker 29.7.2. We deployed WebArena’s Magento storefront and WebShop in local Docker containers, with environment snapshots and task configurations fixed within each comparison.

A *task* or *goal* is a unique specification; an *episode* is one agent attempt under a model and observation condition. Tables report successful episodes over recorded attempts. Results are not pooled across environments, and differing denominators are retained rather than treated as matched pairs.

6.1 Evaluation setup and cases. Five authored domains support comparison of Affora with ARIA and structured-data augmentation, an `agent.md` read layer, and a WebMCP-style action layer. WebArena’s Magento storefront, WebShop, and four independently authored applications test transfer beyond interfaces built for Affora. The release workflow is an author-built adaptation of a community template and is evaluated separately.

Magento provides a controlled transport test: we introduce a component-level deficit into an otherwise functional application and ask whether the published rule restores the lost behavior. WebShop and the four applications contribute deficits already present in their interfaces. Before agent runs, application tasks are classified by whether the target is visible, predictably disclosed by its label, or hidden behind a label that does not predict its contents.

Within each comparison, both arms use the same recorded-state success criterion. Page-level gates assess whether a rewrite preserves the original information, available interactions, and external effects. These gates establish transformation validity, not a guarantee of agent success.

6.2 Comparison with alternative interventions. On the primary comparison set, Affora and the instruction-file condition both reach full completion, an improvement of approximately 23 percentage points over baseline (Table 2). Affora also reaches full completion on the harder set, while the instruction-file condition has one failure. ARIA and structured-data augmentation produces no consistent improvement in these conditions.

The WebMCP-style condition reaches full completion on its applicable action set. Its different denominator means that this result is not a matched comparison with the other interventions. The findings support the value of making required information or actions available through a representation the agent consumes, without establishing that one mechanism is universally superior.

Table 2. **Completion under alternative interventions.** Entries are successful episodes / attempts (rounded percentage). The WebMCP-style action set has different coverage and is shown separately.

Intervention	Primary set	Harder set
Baseline	46/60 (77%)	28/33 (85%)
ARIA and structured data	45/60 (75%)	29/33 (88%)
Instruction file (<code>agent.md</code>)	60/60 (100%)	32/33 (97%)
Affora	60/60 (100%)	33/33 (100%)
WebMCP-style tools (applicable action set)	–	55/55 (100%)

Green marks a favourable change; red an unfavourable change. Black denotes no change or no matched comparison. Colours do not indicate statistical significance.

6.3 Transfer and rule coverage. Across independent interfaces, improvement tracks the presence of the deficit a rule addresses. On Magento, introducing a targeted deficit lowers completion by approximately eight percentage points. Applying the corresponding rewrite restores completion to approximately the original level (Table 3). Rewriting already-sound components produces no improvement.

WebShop distinguishes rule coverage from faithful implementation. The published rules leave its native hidden-radio failure unchanged. Extending the rule set to cover that idiom raises full-reward completion by approximately 61 percentage points. This extension is a diagnostic response to a newly encountered pattern, not evidence that the original rule set already generalized to it.

In the administrative applications, the largest gains occur for targets behind labels that do not predict their contents. Results for predictably labelled groups are mixed, while already-visible targets show little change. MUI and Atomic CRM are near-null at component level. Table 4 reports the application and task-class breakdown, including negative changes. Denominators are recorded attempts and can differ between arms; unmatched attempts are not treated as paired evidence.

6.4 Conformance and workflow efficiency. Conformance. Preservation gates and rule checks answer different questions. The former test whether the task remains equivalent after a rewrite; the latter inspect whether the interface satisfies specified design requirements. Neither establishes that the requirements cover every

Table 3. **Transfer and coverage in independent environments.** Successful episodes / attempts (rounded percentage). Magento’s existing component comparison is separate from the three-model deletion–restoration experiment; WebShop distinguishes the published rules from a subsequent coverage extension.

Comparison	Before	After
Magento: existing component comparison		
Already-sound components	7/40 (18%)	7/40 (18%)
Untouched control tasks	3/40 (8%)	5/40 (13%)
Magento: three-model deletion–restoration		
Magento: introduce targeted deficit	33/216 (15%)	16/216 (7%)
Magento: repair targeted deficit	16/216 (7%)	34/216 (16%)
Unaffected task pairs (287 pairs)	no systematic movement	
WebShop: rule coverage		
WebShop: published rules	9/117 (8%)	9/117 (8%)
WebShop: extend hidden-radio coverage	9/117 (8%)	80/117 (68%)

Green marks a favourable change; red an unfavourable change. Black denotes no change or no matched comparison. Colours do not indicate statistical significance. For the deliberate deficit, green marks the expected decrease.

Table 4. **Transfer by application and target class.** Successful episodes / attempts (rounded percentage). Three models are represented for shadcn-admin, Ant Design Pro, and Atomic CRM; two for the MUI template.

Application / target class	Baseline	Affora
shadcn-admin: 28 task specifications		
Predictably labelled group	70/89 (79%)	66/89 (74%)
Label does not predict target	16/71 (23%)	46/72 (64%)
Already visible	82/88 (93%)	81/90 (90%)
Ant Design Pro: 21 task specifications		
Predictably labelled group	0/79 (0%)	11/78 (14%)
Label does not predict target	12/18 (67%)	18/18 (100%)
Already visible	86/86 (100%)	89/89 (100%)
MUI dashboard template: 20 task specifications		
All target classes	95/120 (79%)	97/120 (81%)
Atomic CRM: 11 task specifications		
All target classes	71/98 (72%)	70/99 (71%)

Green marks a favourable change; red an unfavourable change. Black denotes no change or no matched comparison. Colours do not indicate statistical significance.

interaction pattern or that an agent will complete the task. WebShop illustrates the coverage limit, while near-null component rewrites show why passing component checks cannot establish the quality of an entire workflow. We do not report a separate quantitative validation of the check suite here.

Workflow design. We compare baseline and Affora versions of an author-built release workflow using two models and two observation channels. Correct publication and completion of the final verification are assessed separately. Costs are compared only for paired episodes in which both versions complete the task.

Main result. Affora reduces action counts by approximately 21–40% and total token use by 24–41% across the four model–channel conditions (Table 5). Reasoning-token changes are mixed. Completion improves in

the Luna vision condition and remains unchanged in the others. With only one or two successful pairs per condition, the results provide preliminary evidence of lower interaction cost in this workflow, not a general estimate of efficiency gains.

Table 5. **Release-workflow completion and interaction cost.** Arrows denote baseline $\rightarrow$ Affora. Completion uses all recorded episodes; cost totals use only pairs where both arms succeed. Model-channel conditions are not pooled.

Metric	Vision		DOM/AX	
	Luna	Terra	Luna	Terra
Task completion	1/2 $\rightarrow$ 2/2 (+100%)	2/2 $\rightarrow$ 2/2 (0%)	2/2 $\rightarrow$ 2/2 (0%)	2/2 $\rightarrow$ 2/2 (0%)
Correct publication	2/2 $\rightarrow$ 2/2 (0%)	2/2 $\rightarrow$ 2/2 (0%)	2/2 $\rightarrow$ 2/2 (0%)	2/2 $\rightarrow$ 2/2 (0%)
Successful pairs	1	2	2	2
Actions	50 $\rightarrow$ 34 (-32%)	98 $\rightarrow$ 77 (-21%)	111 $\rightarrow$ 67 (-40%)	108 $\rightarrow$ 77 (-29%)
Total tokens (k)	196.5 $\rightarrow$ 123.7 (-37%)	404.5 $\rightarrow$ 308.0 (-24%)	345.8 $\rightarrow$ 203.2 (-41%)	388.0 $\rightarrow$ 278.1 (-28%)
Reasoning tokens	854 $\rightarrow$ 558 (-35%)	608 $\rightarrow$ 925 (+52%)	2,219 $\rightarrow$ 929 (-58%)	602 $\rightarrow$ 620 (+3%)

Parentheses show percentage change relative to baseline, rounded to whole percentages; 1/2 to 2/2 is +100% relative (+50 percentage points). Costs are totals over successful pairs; total tokens include reasoning tokens. Green marks higher completion or lower cost, red the reverse, and black no change. Colours do not indicate statistical significance.

Visualising the full-workflow case. Figure 8 shows how the interface changes appear along a fixed baseline–Affora trajectory. Both episodes reach correct publication, but only the Affora episode completes final verification. The figure illustrates the recorded interaction sequence; Table 5 provides the aggregate results.

Baseline · conventional workflow

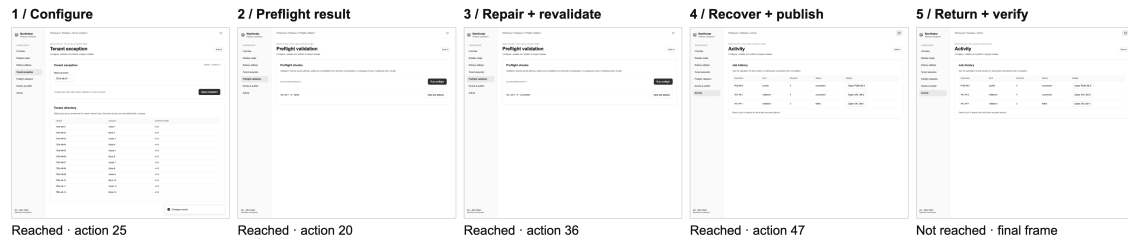


Fig. 8. **From publication to verified completion.** A fixed illustrative pair from the release workflow in Table 5. Five checkpoint categories expose the UI and state-engineering differences; aggregate evidence belongs to the table.

7. Discussion and conclusion

Affora suggests a broader design position: as agents become software operators, interfaces increasingly serve both human and machine readers. Agent-friendly design should therefore improve machine operation without making software less legible, observable, or controllable for people.

7.1 Shared and observable agent experience. Affora treats agent experience as a property of the software environment, not only of the agent. Explicit controls, persistent state, recoverable outcomes, and visible progress help agents operate an interface, but they can also make autonomous activity easier for people to inspect.

This matters because machine actions may occur faster than people can follow them step by step. Human supervision therefore cannot depend only on reading an execution trace. A shared interface can instead expose the consequential state of the work: what changed, what remains incomplete, what failed, and what can be reversed. We treat this supervisability benefit as a design implication rather than a measured claim, since the present work does not include a human study.

7.2 Efficiency as an interface property. Affora also suggests that agent efficiency is partly an interface property. When state, available actions, progress, and recovery information are explicit, an agent can spend fewer interaction cycles rediscovering context, probing hidden controls, or repeating actions whose effects are unclear. In the end-to-end workflow evaluation, Affora reduces both action count and total token use across all four model-channel conditions, although reasoning-token use is mixed.

This shifts part of the efficiency problem from model optimization to interface design. A more legible interface can reduce unnecessary observations, actions, and context consumption even when the underlying agent is unchanged. In deployed systems, this may translate into lower inference cost and latency, although wall-clock savings were not directly measured in our experiments.

7.3 Shared interfaces versus tool and command interfaces. Affora is complementary to structured agent interfaces such as WebMCP, APIs, and command-line tools. These mechanisms can provide more direct and efficient machine action by avoiding visual grounding and GUI operation. Affora addresses a different question: whether task state and interaction meaning remain represented on a surface that both people and agents can inspect.

The two approaches can coexist. Agents may act through structured tools while consequential state is projected back into a shared interface. The same distinction applies to command-line interaction: commands and textual outputs are highly machine-friendly, but a persistent interface can better expose relationships, progress, outstanding work, and recovery options over time.

This suggests a broader principle: machine-efficient action need not require a machine-only representation of state.

7.4 Beyond the DOM: desktop interfaces and software for human-agent collaboration. The semantic substrate studied here is implemented through web representations, but the distinction between visible and machine-readable interface structure is not web-specific. Windows UI Automation exposes control properties and

supported actions (Microsoft 2025). macOS accessibility APIs and Linux AT-SPI likewise expose structured interface information (Apple 2015) (The AT-SPI2 Maintainers n.d.). Electron can expose Chromium’s accessibility tree through platform accessibility support (Electron Contributors n.d.). These mechanisms suggest possible implementations of Affora’s semantic substrate beyond the DOM; transfer to them has not been evaluated here.

The same principles can therefore be asked of desktop software: whether visible controls correspond to machine-operable elements, whether state and available actions are recoverable, and whether task state survives transitions between views or windows. Testing Affora across native and Electron applications would determine how far these rules generalize beyond the web.

Software for human–agent collaboration extends this design problem to one or more people and agents working together through shared interfaces and persistent task state. Explicit state alone may be insufficient when participants act on shared work: the interface may also need to show who made a change, who is responsible for the next step, and whether actions conflict or can be reversed. Supporting this coordination is a possible extension of Affora’s state and continuity rules, but has not been implemented or evaluated here.

7.5 Limitations. The present evidence is bounded in several ways. We evaluate small- and mid-tier models rather than frontier systems, and the strongest visual-design results come from components and relatively short tasks. The independent applications are concentrated in administrative and commerce interfaces, so broader ecological validity remains untested.

The work also evaluates machine readers rather than human users. Accessibility conformance and preservation of familiar interface structure do not establish usability, trust, or supervisability for people. The executable checks likewise cover only properties that can be decided from the rendered artifact; they are a conformance floor rather than a predictor of task success.

Finally, the extensions to desktop interfaces and software for human–agent collaboration discussed above remain hypotheses. The current implementation and evaluation are primarily web-based.

7.6 Conclusion. Affora shows that designing for computer-use agents does not require collapsing software into a machine-oriented interface. Agent-relevant controls, choices, state, and outcomes can be made explicit while visual expression remains flexible.

More broadly, the work points toward software whose state remains legible regardless of who operates it. Agents may act through graphical interfaces, structured tools, command lines, or APIs; what can remain shared is the interface’s representation of what is possible, what has happened, and what state the system is now in. Affora offers one design-system approach toward that shared human–agent surface.

Generative AI Use Disclosure

Generative AI tools were used for language editing, restructuring, and implementation support during manuscript and artifact preparation. AI model APIs were also used to run the reported agent evaluations. The author verified all text, figures, references, code, experimental procedures, and reported results, and retains full responsibility for the work.

References

- Apple. 2015. Accessibility Programming Guide for OS X. <https://developer.apple.com/library/archive/documentation/Accessibility/Conceptual/AccessibilityMacOSX/index.html>.
- The AT-SPI2 Maintainers. n.d. Atspi 2.0 API Reference. Accessed September 11, 2026. <https://gnome.pages.gitlab.gnome.org/at-spi2-core/libatspi/>.
- Beaudouin-Lafon, Michel, Susanne Bødker, and Wendy E. Mackay. 2021. “Generative Theories of Interaction.” *ACM Transactions on Computer-Human Interaction* 28 (6). <https://doi.org/10.1145/3468505>.
- Deng, Xiang, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. “Mind2Web: Towards a Generalist Agent for the Web.” In *Advances in Neural Information Processing Systems 36 (NeurIPS Datasets and Benchmarks)*.
- Dourish, Paul, and Victoria Bellotti. 1992. “Awareness and Coordination in Shared Workspaces.” In *Proceedings of the ACM Conference on Computer-Supported Cooperative Work (CSCW ’92)*, 107–14. ACM.
- Electron Contributors. n.d. Accessibility. Electron Documentation. Accessed September 11, 2026. <https://www.electronjs.org/docs/latest/tutorial/accessibility>.
- Gibson, James J. 1977. The Theory of Affordances. In *Perceiving, Acting, and Knowing: Toward a New Psychology of Public and Private*, edited by Robert Shaw and John Bransford, 67–82. Holt, Rinehart and Winston.
- Goldenberg, Dmitri, and Yulia Goldenberg. 2025. Agent Experience: Nielsen’s Usability Heuristics Analysis for GenAI Agents. In *GenAICHI: CHI 2025 Workshop on Generative AI and HCI*. https://generativeaiandhci.github.io/papers/2025/genaichi2025_5.pdf.
- He, Hongliang, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. “WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models.” In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Howard, Jeremy. 2024. “The /Llms.txt File.” <https://llmstxt.org/>.
- Koh, Jing Yu, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. 2024. “VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks.” In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Lamine, Yassine, and Jinghui Cheng. 2022. “Understanding and Supporting the Design Systems Practice.” *Empirical Software Engineering* 27 (146).
- Le Sellier De Chezelles, Thibault, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, et al. 2024. “The BrowserGym Ecosystem for Web Agent Research.” *arXiv Preprint arXiv:2412.05467*.
- Liu, Jiateng, Rushi Wang, Bingxuan Li, Kunlun Zhu, Yifan Shen, Qingyun Wang, Ahmed Abbasi, Denghui Zhang, and Heng Ji. 2026. Augmenting Interface Usability Heuristics for Reliable Computer-Use Agents. *arXiv:2605.02729*. <https://doi.org/10.48550/arXiv.2605.02729>.
- Lù, Xing Han, Zdeněk Kasner, and Siva Reddy. 2024. “WebLINX: Real-World Website Navigation with Multi-Turn Dialogue.” In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. Vol. 235. PMLR.

- Microsoft. 2025. UI Automation Overview. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/winauto/uiauto-uiautomationoverview>.
- Miller, George A. 1956. The Magical Number Seven, Plus or Minus Two: Some Limits on Our Capacity for Processing Information. *Psychological Review* 63 (2): 81–97.
- Nielsen, Jakob. 1994. Enhancing the Explanatory Power of Usability Heuristics. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 152–158. ACM.
- Norman, Donald A. 2008. “Signifiers, Not Affordances.” *Interactions* 15 (6): 18–19.
- Norman, Donald A. 2013. *The Design of Everyday Things*. Revised and expanded. Basic Books.
- Putnam, Cynthia, Emma J. Rose, and Craig M. MacDonald. 2023. “‘It Could Be Better. It Could Be Much Worse’: Understanding Accessibility in User Experience Practice with Implications for Industry and Education.” *ACM Transactions on Accessible Computing*.
- Reid, Blake E. 2024. “The Curb-Cut Effect and the Perils of Accessibility Without Disability.” In *Feminist Cyberlaw*, edited by Meg Leta Jones and Amanda Levendowski, 104–16. University of California Press.
- Rongon, Rabab Khan, Nazmul Hasan, and Sadab Khan Prangon. 2026. Designing Agent-Readable Interfaces for Human-AI-UI Collaboration. Position paper, *CHI 2026 Workshop on Human-AI-UI Interactions Across Modalities*. Author-posted manuscript. https://www.researchgate.net/publication/404610890_Designing_Agent-Readable_Interfaces_for_Human-AI-UI_Collaboration.
- Shneiderman, Ben, Catherine Plaisant, Maxine Cohen, Steven Jacobs, Niklas Elmqvist, and Nicholas Diakopoulos. 2016. *Designing the User Interface: Strategies for Effective Human-Computer Interaction*. 6th ed. Pearson.
- W3C Web Accessibility Initiative. 2018. “Web Content Accessibility Guidelines (WCAG) 2.1.” W3C Recommendation.
- Walderman, Brandon, Leo Lee, Andrew Nolan, David Bokan, Khushal Sagar, and Hannah Van Opstal. 2026. “WebMCP: Exposing Web Application Functionality as Agent Tools.” W3C Web Machine Learning Community Group.
- Yang, Jianwei, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. 2023. “Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V.” arXiv:2310.11441.
- Yao, Shunyu, Howard Chen, John Yang, and Karthik Narasimhan. 2022. “WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents.” In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. <https://doi.org/10.52202/068431-1508>.
- Zheng, Boyuan, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. “GPT-4V(ision) Is a Generalist Web Agent, If Grounded.” In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. Vol. 235. PMLR.
- Zhou, Shuyan, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, et al. 2024. “WebArena: A Realistic Web Environment for Building Autonomous Agents.” In *International Conference on Learning Representations (ICLR)*.

Appendix

Principle index

The following reference table separates each rule from its associated check and evidence status. A measured effect applies to the tested condition; observed and proposed requirements are not presented as experimentally validated. Layout applies P5 and P9 across page composition rather than adding a separate principle family. In the check column, S1–S8 are component checks, FC1–FC8 flow checks, K1–K3 site checks, and FP1–FP5 flow patterns.

evidence	principle	scope	deciding check / implementation
measured	P1 State is stated	component	S4, S7
measured	P2 Presence over disclosure	component	S5
measured	P3 Every affordance is named, in text	component	S1, S2
measured	P7 Feedback persists	component	S7 · FP4
measured, null	P8 One concept, one word	component + site	S2, K2
measured	F1 Depth is the price	flow	FC1 · FP1
measured	F2 Every screen states its own state	flow	FC2, FC3 · FP3
measured / proposed	F4 Reversal over confirmation	flow	FC5 · FP5
measured	F5 Dual paths: guided and direct	flow	FC6 · FP1
measured	F6 Failures name their remedy	flow	FC7 · FP4
measured	SC2 One glyph per function, name travels	site	K2
observed	P4 Meaning never rests on colour or geometry alone	component	S8
observed	P5 Visual hierarchy mirrors semantic hierarchy	component + layout	S3
observed	P9 Structural isomorphism	component + layout	S3, S5
observed	P10 Destructive actions are marked	component	FC5 · FP5
null	P6 Constraints are stated before the attempt	component + flow	S3, FC4 · FP2
null	F3 Constraints precede attempts	flow	FC4 · FP2
proposed	F7 Completion is stated	flow	FC8
proposed	SC1 Stable semantic colour roles	site	K1
null with history	SC3 Navigation consistency	site	K3